

Module 4: Node.js, Express & TypeORM > Section 11: Entities, Relations & CRUD with TypeORM > 11-03: CRUD Operations with TypeORM

CRUD Operations with TypeORM

Section	11 — Entities, Relations, and CRUD with TypeORM
Subtopic	11-03 — CRUD Operations
Duration	~2–3 hours (within the 6–8 hr section allocation)
Tags	Theory Practical TypeScript
Topics covered	Entity Manager, Repository API, save / find / update / remove / delete, Custom Repositories, class-validator integration
Prerequisites	11-01 (Entity Definition), 11-02 (Relations & Eager/Lazy Loading), 10-xx (DataSource setup)

Learning Objectives — This Subtopic

- Distinguish between the **Entity Manager** and the **Repository** pattern and identify when to use each.
- Perform all four CRUD operations using TypeORM's repository methods (`save` , `find` , `findOne` , `update` , `remove` , `delete`) in a TypeScript project.
- Use **FindOptions** (`where` , `select` , `order` , `relations` , `take` , `skip`) to shape query results.
- Create a **Custom Repository** by extending `Repository<T>` and encapsulating domain-specific query logic.
- Integrate **class-validator** decorators on entities and trigger validation before persistence.
- Handle and surface TypeORM errors cleanly within an Express route handler.

1. Entity Manager vs Repository — Choosing the Right Tool

TypeORM provides two overlapping APIs for interacting with the database: the **Entity Manager** and **Repositories**. Both ultimately issue SQL, but they differ in scope and ergonomics.

1.1 Entity Manager

The `EntityManager` is a single, global interface that can operate on *any* registered entity. You access it via `dataSource.manager`. It is useful when one unit of work touches multiple unrelated entity types and you want a single handle rather than many repositories.

src/index.ts

```
import "reflect-metadata";
import { DataSource } from "typeorm";
import { User } from "../entity/User";

const AppDataSource = new DataSource({
  type: "sqlite",
  database: "db.sqlite",
  synchronize: true,
  logging: false,
  entities: [User],
});

AppDataSource.initialize().then(async () => {
  const manager = AppDataSource.manager;

  // Create and persist a User via EntityManager
  const user = manager.create(User, { name: "Alice", email: "alice@example.com" });
  await manager.save(user);

  console.log("Saved user with id:", user.id);
}).catch(console.error);
```

Output:

Saved user with id: 1

What is happening? `manager.create(User, {...})` instantiates a `User` entity and assigns the provided fields without touching the database. `manager.save(user)` then issues an `INSERT` (or `UPDATE` if the entity already has a primary key). The `id` is populated by TypeORM after the insert completes.

1.2 Repository

A `Repository<T>` is a scoped version of the Entity Manager — it is bound to a single entity type. You obtain one via `dataSource.getRepository(EntityClass)`. In practice, repositories are the preferred pattern for Express service classes because each service owns a single entity type and the API is slightly more concise.

Analogy: Think of the Entity Manager as a department store — you can buy from any section. A Repository is a dedicated shop that stocks only one product line. The dedicated shop's staff know that product deeply and the API reflects it.

src/index.ts

```
import "reflect-metadata";
import { DataSource } from "typeorm";
import { User } from "../entity/User";

const AppDataSource = new DataSource({
  type: "sqlite",
  database: "db.sqlite",
  synchronize: true,
  logging: false,
  entities: [User],
});

AppDataSource.initialize().then(async () => {
  const userRepository = AppDataSource.getRepository(User);

  const user = userRepository.create({ name: "Bob", email: "bob@example.com" });
  await userRepository.save(user);

  console.log("Saved user with id:", user.id);
}).catch(console.error);
```

Output:

Saved user with id: 1

Key Insight: `getRepository(User)` returns a new `Repository` instance each time but it is lightweight — there is no performance cost to calling it multiple times. In a real Express application you typically call it once per service class constructor or inject a shared instance.

2. The Reference Entity

All examples in this document use the following `User` entity, which you should have defined in Section 11-01. It is reproduced here for reference.

src/entity/User.ts

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn } from "typeorm";

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ length: 100 })
  name: string;

  @Column({ unique: true })
  email: string;

  @Column({ default: true })
  isActive: boolean;

  @CreateDateColumn()
  createdAt: Date;
}
```

No output is produced for an entity file — it is a configuration artefact, not a runnable script. The next runnable example will use this entity.

3. Create — Persisting New Records

The canonical way to insert a new row is the two-step `create` + `save` pattern. The `create` call instantiates the entity in memory so that TypeORM's lifecycle hooks and subscriber events fire correctly.

Warning: Never instantiate an entity with `new User()` and pass it directly to `save`. TypeORM will not recognise it as a managed entity instance and lifecycle hooks will not fire. Always use `repository.create({...})`.

3.1 Inserting a Single Record

STAGE 1

src/crud-demo.ts

```
import "reflect-metadata";
import { AppDataSource } from "../data-source";
import { User } from "../entity/User";

async function main(): Promise<void> {
  await AppDataSource.initialize();
  const repo = AppDataSource.getRepository(User);

  const user = repo.create({ name: "Carol", email: "carol@example.com" });
  const saved = await repo.save(user);

  console.log("Inserted:", saved);
}

main().catch(console.error);
```

Output:

```
Inserted: User {
  id: 1,
  name: 'Carol',
  email: 'carol@example.com',
  isActive: true,
  createdAt: 2026-03-01T09:00:00.000Z
}
```

After `save` resolves, the returned object is the same entity instance with its `id` and `createdAt` fields populated. TypeORM issues a single `INSERT` statement.

3.2 Inserting Multiple Records at Once

STAGE 2

`save` also accepts an array. TypeORM batches the inserts into as few SQL statements as the driver supports.

src/crud-demo.ts

```
const users = repo.create([
  { name: "Dave", email: "dave@example.com" },
  { name: "Eve", email: "eve@example.com" },
]);

const saved = await repo.save(users);
saved.forEach(u => console.log(`id=${u.id} name=${u.name}`));
```

Output:

```
id=2 name=Dave
id=3 name=Eve
```

Key Insight: `save` performs an upsert by primary key. If the entity already has an `id` that matches an existing row, TypeORM issues an `UPDATE`. If the entity has no `id` (or the id does not exist in the database), it issues an `INSERT`. This dual behaviour is deliberate and consistent across all TypeORM drivers.

4. Read — Querying Records

TypeORM's repository exposes several read methods. The right one depends on how many records you expect and what conditions you need.

4.1 find — Return All Matching Rows

STAGE 1

Called without arguments, `find` selects every row in the table.

src/crud-demo.ts

```
const all = await repo.find();
console.log("Total users:", all.length);
```

Output:

Total users: 3

4.2 find with FindOptions

STAGE 2

`find` accepts a `FindManyOptions<T>` object. The most commonly used keys are shown below.

src/crud-demo.ts

```
import { ILike } from "typeorm";

const activeUsers = await repo.find({
  where: { isActive: true },
  select: { id: true, name: true, email: true },
  order: { name: "ASC" },
  skip: 0,
  take: 10,
});

activeUsers.forEach(u => console.log(u.name, u.email));
```

Output:

```
Carol carol@example.com
Dave dave@example.com
Eve eve@example.com
```

The `FindManyOptions` breakdown:

- `where` — a plain object or a TypeORM operator expression such as `ILike`, `Between`, `In`. Multiple keys within the same object are combined with `AND`.
- `select` — restricts which columns are returned. Accepts an object of booleans keyed by entity property name.
- `order` — an object mapping column names to `"ASC"` or `"DESC"`.
- `skip` / `take` — offset and limit, used for pagination.

4.3 findOne — Return a Single Row

STAGE 3

`findOne` accepts `FindOneOptions<T>`, which is the same shape as `FindManyOptions` but returns the first match or `null`.

```
src/crud-demo.ts
```

```
const user = await repo.findOne({ where: { email: "carol@example.com" } });

if (user) {
  console.log("Found:", user.name);
} else {
  console.log("User not found");
}
```

Output:

```
Found: Carol
```


Important: `findOne` returns `null` when no row matches — it does not throw. Always check the return value before using it. The older `findOneOrFail` method throws an `EntityNotFoundError` if nothing matches, which is useful when the absence of a record is genuinely exceptional.

4.4 Loading Relations

When a `User` has related entities (covered in Section 11-02), you can load them in the same query using the `relations` option rather than issuing a second query.

src/crud-demo.ts

```
// Assuming User has a @OneToMany(() => Post, ...) posts relation
const userWithPosts = await repo.findOne({
  where: { id: 1 },
  relations: { posts: true },
});

console.log("Post count:", userWithPosts?.posts.length ?? 0);
```

Output:

Post count: 2

Key Insight: The `relations` option issues a `LEFT JOIN` under the hood. For deeply nested relations you can use nested objects: `{ posts: { comments: true } }`.

5. Update — Modifying Existing Records

There are two distinct update pathways: the `save` method (entity-centric) and the `update` method (SQL-centric). They serve different purposes.

5.1 Update via save

STAGE 1

The recommended approach for full entity updates: fetch the entity, mutate it in memory, then save it back. TypeORM detects that the entity already has a primary key and issues an `UPDATE` .

src/crud-demo.ts

```
const user = await repo.findOne({ where: { id: 1 } });

if (user) {
  user.name = "Carol Smith";
  await repo.save(user);
  console.log("Updated name:", user.name);
}
```

Output:

Updated name: Carol Smith

This approach triggers TypeORM subscriber events and ensures that nullable columns are not accidentally overwritten with `undefined` .

5.2 Update via repository.update

STAGE 2

`repository.update(criteria, partialEntity)` issues a direct SQL `UPDATE` without loading the entity first. It is more performant when you need to modify a single column and do not need lifecycle hooks.

src/crud-demo.ts

```
import { UpdateResult } from "typeorm";

const result: UpdateResult = await repo.update(
  { id: 1 }, // criteria (WHERE clause)
  { isActive: false } // partial entity (SET clause)
);

console.log("Rows affected:", result.affected);
```

Output:

Rows affected: 1

Important: `repository.update` does not return the updated entity — it returns an `UpdateResult` with metadata about the operation. If you need the updated entity, call `findOne` after the update. Additionally, this method does not trigger TypeORM subscriber `BeforeUpdate` / `AfterUpdate` events because no entity instance is loaded.

6. Delete — Removing Records

TypeORM provides `remove` and `delete` for deletion, mirroring the duality seen in updates.

6.1 remove — Entity-Level Deletion

STAGE 1

`remove` accepts a managed entity instance and issues a `DELETE` by primary key. The entity's `id` is set to `undefined` after deletion to signal that it is no longer persisted.

src/crud-demo.ts

```
const user = await repo.findOne({ where: { id: 3 } });

if (user) {
  const removed = await repo.remove(user);
  console.log("Removed user; id is now:", removed.id); // undefined
}
```

Output:

Removed user; id is now: undefined

6.2 delete — Criteria-Level Deletion

STAGE 2

`delete` accepts criteria (a plain object or a primary key value) and deletes without loading an entity. It is the performant choice when you know the identifier and do not need lifecycle hooks.

src/crud-demo.ts

```
import { DeleteResult } from "typeorm";

const result: DeleteResult = await repo.delete({ id: 2 });
console.log("Rows deleted:", result.affected);
```

Output:

Rows deleted: 1

You can also pass a primary key value directly: `repo.delete(2)` — TypeORM maps it to the `@PrimaryGeneratedColumn` column automatically.

Key Insight: Use `remove` when you have the entity in memory and need lifecycle hooks. Use `delete` when you only have the identifier and want the fastest path to deletion.

7. Auxiliary Read Methods — count and exists

Two additional methods are frequently useful without involving a full entity fetch.

7.1 count

src/crud-demo.ts

```
const total = await repo.count({ where: { isActive: true } });
console.log("Active users:", total);
```

Output:

Active users: 2

7.2 exists

Introduced in TypeORM 0.3.x, `exists` returns a boolean and issues a more efficient

`SELECT 1 ... LIMIT 1` rather than fetching full rows.

src/crud-demo.ts

```
const taken = await repo.exists({ where: { email: "carol@example.com" } });  
console.log("Email already taken:", taken);
```

Output:

Email already taken: true

8. FindOperators — Expressive Criteria

TypeORM exports operator functions from the `typeorm` package that map to SQL expressions, allowing you to write readable criteria without dropping to raw SQL.

src/crud-demo.ts

```
import { ILike, In, Between, Not, IsNull } from "typeorm";  
  
// Users whose name contains "carol" (case-insensitive)  
const byName = await repo.find({ where: { name: ILike("%carol%") } });  
  
// Users with specific ids  
const byIds = await repo.find({ where: { id: In([1, 2]) } });  
  
// Users created in a date range (omit for brevity; Date objects required)  
// const recent = await repo.find({ where: { createdAt: Between(startDate, endDate) } });
```

```
// Users that are NOT inactive
const notInactive = await repo.find({ where: { isActive: Not(false) } });

// Users with a null field (useful for optional columns)
// const noAvatar = await repo.find({ where: { avatarUrl: IsNull() } });

console.log("By name match:", byName.length);
console.log("By ids [1,2]:", byIds.length);
console.log("Not inactive:", notInactive.length);
```

Output:

```
By name match: 1
By ids [1,2]: 2
Not inactive: 2
```

9. Custom Repositories

The built-in repository API covers generic operations well. Business-specific queries — for example, "find all users who have not logged in for 30 days" — do not belong scattered across route handlers. Custom repositories centralise that logic and make it testable in isolation.

9.1 Extending Repository<T>

Create a class that extends `Repository<User>` and annotate it with no special decorator. You then bind it to the `DataSource` using `dataSource.getRepository(User).extend({...})`.

STAGE 1 — DEFINE THE CUSTOM REPOSITORY

src/repository/UserRepository.ts

```
import { Repository, DataSource } from "typeorm";
import { User } from "../entity/User";

export class UserRepository extends Repository<User> {

  async findActiveByName(name: string): Promise<User[]> {
    return this.find({
      where: { name: ILike(`%${name}%`), isActive: true },
      order: { createdAt: "DESC" },
    });
  }
}
```

```
    });  
  }  
  
  async deactivateUser(id: number): Promise<void> {  
    await this.update({ id }, { isActive: false });  
  }  
  
  async existsByEmail(email: string): Promise<boolean> {  
    return this.exists({ where: { email } });  
  }  
}
```

STAGE 2 — BIND IT TO THE DATASOURCE

src/data-source.ts

```
import "reflect-metadata";  
import { DataSource } from "typeorm";  
import { User } from "../entity/User";  
  
export const AppDataSource = new DataSource({  
  type: "sqlite",  
  database: "db.sqlite",  
  synchronize: true,  
  logging: false,  
  entities: [User],  
});  
  
export function getUserRepository() {  
  return AppDataSource.getRepository(User).extend(UserRepository.prototype);  
}
```

Key Insight: The `.extend()` method (TypeORM 0.3+) is the recommended approach for custom repositories. It avoids the deprecated `@EntityRepository` decorator that was removed in TypeORM 0.3. The returned object has all the built-in `Repository` methods *plus* your custom methods.

STAGE 3 — USE IN APPLICATION CODE

src/index.ts

```
import { AppDataSource, getUserRepository } from "../data-source";  
import { ILike } from "typeorm";  
import { UserRepository } from "../repository/UserRepository";  
  
AppDataSource.initialize().then(async () => {
```

```
const userRepo: UserRepository = getUserRepository() as UserRepository;

const carols = await userRepo.findActiveByName("carol");
console.log("Active Carols:", carols.map(u => u.name));

const emailTaken = await userRepo.existsByEmail("dave@example.com");
console.log("dave@example.com taken:", emailTaken);

await userRepo.deactivateUser(2);
console.log("Deactivated user 2");
}).catch(console.error);
```

Output:

```
Active Carols: ['Carol Smith']
dave@example.com taken: true
Deactivated user 2
```

10. Validation with class-validator

TypeORM does not validate entity data before persisting it — column constraints (`unique` , `length`) are enforced at the database level, which produces database-driver errors rather than friendly application errors. The `class-validator` library lets you declare validation rules directly on entity properties with decorators and validate them before the `save` call reaches the database.

10.1 Installation

Run: `npm install class-validator class-transformer`

`class-transformer` is required alongside `class-validator` because it handles the transformation of plain objects into class instances, which is necessary for the validators to fire.

10.2 Annotating the Entity

```
src/entity/User.ts
```



```
import {
  Entity, PrimaryGeneratedColumn, Column, CreateDateColumn
} from "typeorm";
import {
  IsEmail, IsNotEmpty, Length, IsBoolean
} from "class-validator";

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ length: 100 })
  @IsNotEmpty({ message: "Name must not be empty" })
  @Length(2, 100, { message: "Name must be between 2 and 100 characters" })
  name: string;

  @Column({ unique: true })
  @IsEmail({}, { message: "Must be a valid email address" })
  email: string;

  @Column({ default: true })
  @IsBoolean()
  isActive: boolean;

  @CreateDateColumn()
  createdAt: Date;
}
```

10.3 Validating Before Save

The `validate` function from `class-validator` returns a promise that resolves to an array of `ValidationError` objects. An empty array means all checks passed.

STAGE 1 — VALIDATE AND PERSIST

src/crud-demo.ts

```
import { validate } from "class-validator";
import { AppDataSource } from "../data-source";
import { User } from "../entity/User";

async function createUser(name: string, email: string): Promise<User | string[]> {
  const repo = AppDataSource.getRepository(User);
  const user = repo.create({ name, email });
```

```
const errors = await validate(user);

if (errors.length > 0) {
  const messages = errors.flatMap(e => Object.values(e.constraints ?? {}));
  return messages; // caller decides how to surface these
}

return repo.save(user);
}

AppDataSource.initialize().then(async () => {
  // Valid case
  const result1 = await createUser("Fiona", "fiona@example.com");
  console.log("Valid:", result1);

  // Invalid case - bad email, name too short
  const result2 = await createUser("F", "not-an-email");
  console.log("Errors:", result2);
}).catch(console.error);
```

Output:

```
Valid: User {
  id: 4,
  name: 'Fiona',
  email: 'fiona@example.com',
  isActive: true,
  createdAt: 2026-03-01T09:05:00.000Z
}
Errors: [
  'Name must be between 2 and 100 characters',
  'Must be a valid email address'
]
```

10.4 Validation in an Express Route

The pattern above composes naturally with an Express route. The route handler calls the service function and converts the validation messages to a `400 Bad Request` response.

STAGE 2 — EXPRESS INTEGRATION

src/routes/userRoutes.ts

```
import { Router, Request, Response } from "express";
import { validate } from "class-validator";
import { AppDataSource } from "../data-source";
import { User } from "../entity/User";

const router = Router();
const repo = () => AppDataSource.getRepository(User);

// POST /users - create a user
router.post("/", async (req: Request, res: Response) => {
  const user = repo().create(req.body as Partial<User>);
  const errors = await validate(user);

  if (errors.length > 0) {
    const messages = errors.flatMap(e => Object.values(e.constraints ?? {}));
    res.status(400).json({ errors: messages });
    return;
  }

  try {
    const saved = await repo().save(user);
    res.status(201).json(saved);
  } catch (err: any) {
    // SQLITE_CONSTRAINT_UNIQUE or similar driver errors
    if (err.code === "SQLITE_CONSTRAINT") {
      res.status(409).json({ error: "Email already registered" });
    } else {
      res.status(500).json({ error: "Internal server error" });
    }
  }
});

export default router;
```

Output:

```
# HTTP 400 - invalid payload
POST /users body: { "name": "X", "email": "bad" }
{
  "errors": [
    "Name must be between 2 and 100 characters",
    "Must be a valid email address"
  ]
}

# HTTP 201 - success
```

```
POST /users body: { "name": "Grace", "email": "grace@example.com" }
{
  "id": 5,
  "name": "Grace",
  "email": "grace@example.com",
  "isActive": true,
  "createdAt": "2026-03-01T09:10:00.000Z"
}
```

Key Insight: `class-validator` runs *application-level* validation before the SQL statement is issued. Database constraints are a second safety net. Relying solely on database errors for validation produces hard-to-read driver-specific error codes; combining both layers is best practice.

11. Putting It Together — Full CRUD Route Set

The following demonstrates a complete CRUD route file for the `User` entity, combining all techniques from this document.

src/routes/userRoutes.ts

```
import { Router, Request, Response } from "express";
import { validate } from "class-validator";
import { AppDataSource } from "../data-source";
import { User } from "../entity/User";

const router = Router();
const repo = () => AppDataSource.getRepository(User);

// GET /users - List all active users
router.get("/", async (_req: Request, res: Response) => {
  const users = await repo().find({
    where: { isActive: true },
    select: { id: true, name: true, email: true },
    order: { name: "ASC" },
  });
  res.json(users);
});

// GET /users/:id - get single user
```

```
router.get("/:id", async (req: Request, res: Response) => {
  const user = await repo().findOne({ where: { id: Number(req.params.id) } });
  if (!user) {
    res.status(404).json({ error: "User not found" });
    return;
  }
  res.json(user);
});

// POST /users - create
router.post("/", async (req: Request, res: Response) => {
  const user = repo().create(req.body as Partial<User>);
  const errors = await validate(user);

  if (errors.length > 0) {
    res.status(400).json({ errors: errors.flatMap(e => Object.values(e.constraints ?? {})) });
    return;
  }

  try {
    const saved = await repo().save(user);
    res.status(201).json(saved);
  } catch (err: any) {
    if (err.code === "SQLITE_CONSTRAINT") {
      res.status(409).json({ error: "Email already registered" });
    } else {
      res.status(500).json({ error: "Internal server error" });
    }
  }
});

// PATCH /users/:id - partial update
router.patch("/:id", async (req: Request, res: Response) => {
  const id = Number(req.params.id);
  const user = await repo().findOne({ where: { id } });

  if (!user) {
    res.status(404).json({ error: "User not found" });
    return;
  }

  // Merge only supplied fields onto the existing entity
  Object.assign(user, req.body);

  const errors = await validate(user);
  if (errors.length > 0) {
    res.status(400).json({ errors: errors.flatMap(e => Object.values(e.constraints ?? {})) });
  }
});
```

```

    return;
  }

  const updated = await repo().save(user);
  res.json(updated);
});

// DELETE /users/:id - hard delete
router.delete("/:id", async (req: Request, res: Response) => {
  const result = await repo().delete({ id: Number(req.params.id) });

  if (result.affected === 0) {
    res.status(404).json({ error: "User not found" });
    return;
  }

  res.status(204).send();
});

export default router;

```

Output:

Example responses

```

GET    /users      → 200  [{ "id":1, "name":"Carol Smith", "email":"carol@example.com" }, ...]
GET    /users/99   → 404  { "error": "User not found" }
POST   /users      → 201  { "id":6, "name":"Hana", ... }
PATCH /users/1   → 200  { "id":1, "name":"Carol A. Smith", ... }
DELETE /users/1   → 204  (no body)
DELETE /users/999 → 404  { "error": "User not found" }

```

Summary Table

Concept	Key Point
Entity Manager	Global handle for any entity; obtained via <code>dataSource.manager</code> . Useful for multi-entity units of work.

Concept	Key Point
Repository	Entity-scoped handle; obtained via <code>dataSource.getRepository(Entity)</code> . Preferred pattern for service classes.
create + save	Two-step insert: <code>create</code> instantiates the entity in memory; <code>save</code> issues the INSERT (or UPDATE if an id exists).
find / findOne	Read methods accepting <code>FindManyOptions</code> / <code>FindOneOptions</code> for filtering, projection, ordering, and pagination.
FindOperators	Exported functions (<code>ILike</code> , <code>In</code> , <code>Between</code> , <code>Not</code> , <code>IsNull</code>) that compose SQL expressions within <code>where</code> clauses.
repository.update	Direct SQL UPDATE by criteria; performant but does not load the entity or fire lifecycle hooks.
remove vs delete	<code>remove</code> takes a managed entity (fires hooks); <code>delete</code> takes criteria (no load, no hooks).
count / exists	Aggregate helpers; <code>exists</code> issues a more efficient query than fetching full rows just to check presence.
Custom Repository	Extends <code>Repository<T></code> ; bound via <code>.extend()</code> . Centralises domain-specific queries away from route handlers.
class-validator	Decorators on entity properties; <code>validate(entity)</code> returns <code>ValidationError[]</code> . Run before <code>save</code> for friendly error messages.

Interview Questions

1. What is the difference between `repository.save` and `repository.update` ?

Expected answer: `save` operates on a managed entity instance — it fires lifecycle hooks, triggers subscriber events, and performs an upsert (INSERT if no id, UPDATE if id present). `update` issues a direct SQL `UPDATE` by criteria without loading the entity, so it is faster but does not fire hooks. Use `update` for simple field changes where you do not need the entity loaded; use `save` when hooks matter or when you are working with an already-loaded entity.

2. Why should you use `repository.create({...})` rather than `new User()` ?

Expected answer: `repository.create` returns a fully managed entity instance that TypeORM tracks internally, ensuring that lifecycle hooks (`BeforeInsert` , `AfterInsert` , etc.) and subscriber events fire correctly. An instance created with `new` is not registered with TypeORM's identity map and may bypass these hooks. It also applies default values defined on the entity.

3. What does `findOne` return when no matching row exists?

Expected answer: It returns `null` . It does not throw. If the absence of a row is exceptional and you want an error thrown automatically, use `findOneOrFail` , which throws `EntityNotFoundError` .

4. How would you load a related entity (`Post[]`) alongside a `User` without a second query?

Expected answer: Pass the `relations` option to `findOne` or `find` : `{ relations: { posts: true } }` . TypeORM issues a `LEFT JOIN` to load the related data in a single round-trip. Alternatively, the relation can be marked as eager on the entity definition (covered in Section 11-02) to load it automatically every time.

5. What is the purpose of the `class-validator` library in a TypeORM project?

Expected answer: TypeORM does not validate data before persisting it — database constraints produce low-level driver errors. `class-validator` lets you declare validation rules as decorators on entity properties and call `validate(entity)` before `save` , returning structured `ValidationError` objects that can be surfaced as user-friendly error messages in API responses.

6. How do you create a Custom Repository in TypeORM 0.3+?

Expected answer: Define a class that extends `Repository<T>` with the extra methods, then bind it using `dataSource.getRepository(Entity).extend(CustomRepository.prototype)`. The deprecated `@EntityRepository` decorator was removed in 0.3; the `.extend()` pattern is the current recommended approach.

7. What is the difference between `remove` and `delete`, and when would you choose each?

Expected answer: `remove` accepts a managed entity instance, issues the DELETE by primary key, and fires `BeforeRemove` / `AfterRemove` subscriber events. The entity's `id` becomes `undefined` afterwards. `delete` accepts criteria (plain object or primary key value) and issues a direct SQL DELETE without loading the entity or firing hooks. Choose `remove` when you need hooks; choose `delete` for simple, performant deletion when you only have the identifier.

8. How do you perform a case-insensitive partial name search using `find`?

Expected answer: Import the `ILike` operator from `typeorm` and use it in the `where` clause:
`repo.find({ where: { name: ILike('%carol%') } })`. `ILike` maps to `ILIKE` in PostgreSQL or a `LIKE LOWER()` expression in databases that do not natively support it.

Practical Assignment — Product Inventory CRUD API

Objective: Build a REST API with full CRUD operations for a `Product` entity, including class-validator validation and a custom repository method.

Estimated time: 45–60 minutes

Setup

Create a new project and install dependencies:

```
mkdir product-api && cd product-api
```

```
npm init -y
```

```
npm install express typeorm reflect-metadata sqlite3 class-validator class-transformer
```

```
npm install -D typescript @types/express @types/node ts-node
```

```
npx tsc --init --experimentalDecorators --emitDecoratorMetadata --strict false
```

```
product-api/
├─ src/
│  ├─ entity/
│  │   └─ Product.ts
│  ├─ repository/
│  │   └─ ProductRepository.ts
│  ├─ routes/
│  │   └─ productRoutes.ts
│  ├─ data-source.ts
│  └─ index.ts
├─ package.json
└─ tsconfig.json
```

Steps

1. Define the **Product** entity in `src/entity/Product.ts` with the following columns and validators:
 - `id` — auto-generated primary key
 - `name` — string, not empty, length 2–150
 - `sku` — string, unique, not empty
 - `price` — number (use `@Column('decimal', { precision: 10, scale: 2 })`), must be greater than 0 (`@Min(0.01)` from `class-validator`)
 - `stock` — integer, default 0, minimum 0 (`@Min(0)`)
 - `createdAt` — auto-populated creation timestamp
2. Create a **ProductRepository** in `src/repository/ProductRepository.ts` extending `Repository<Product>` with a custom method `findLowStock(threshold: number): Promise<Product[]>` that returns all products where `stock <= threshold`, ordered by stock ascending.

3. Create the `DataSource` in `src/data-source.ts` with `SQLite`, `synchronize: true`, and export a `getProductRepository()` helper that returns your custom repository bound via `.extend()`.
4. Build the route file in `src/routes/productRoutes.ts` with the following endpoints:
 - `GET /products` — list all products, sorted by name
 - `GET /products/:id` — fetch single product or 404
 - `GET /products/low-stock/:threshold` — use your custom repository method
 - `POST /products` — create with validation; return 400 on invalid input, 409 on duplicate SKU
 - `PATCH /products/:id` — partial update with validation
 - `DELETE /products/:id` — hard delete; return 204 or 404
5. Wire everything in `src/index.ts`: initialise the `DataSource`, mount the router at `/products`, and start listening on port 3000.
6. Test with curl or a REST client:

```
POST /products body: { "name": "Widget A", "sku": "WGT-001", "price": 9.99,
"stock": 50 }
GET /products/low-stock/10
PATCH /products/1 body: { "stock": 5 }
DELETE /products/1
```

Expected Output (seed + low-stock query)

Output:

```
POST /products → 201
{ "id": 1, "name": "Widget A", "sku": "WGT-001", "price": "9.99", "stock": 50, "createdAt": "..." }

POST /products → 201
{ "id": 2, "name": "Gadget B", "sku": "GDG-002", "price": 4.49, "stock": 3, "createdAt": "..." }

GET /products/low-stock/10 → 200
[{ "id": 2, "name": "Gadget B", "sku": "GDG-002", "price": "4.49", "stock": 3 }]

DELETE /products/99 → 404
{ "error": "Product not found" }
```

Bonus Challenge: Add a `GET /products` query parameter `?search=` that filters by product name using `ILike`. For example, `GET /products?search=widget` should return all products whose name contains "widget" (case-insensitive). Combine this with `order` and `take / skip` query parameters to build a paginated search endpoint.

